1. **which file is the user-level library wrapper function int getpid_super(int level offset, int * pid ptr) defined?**

   getpid_super() is defined in <user.h> file, and the actual user-level wrapper that triggers the system call is defined in <usys.S>.

2. **Explain (with the actual code) how the above wrapper function triggers the system call.**

   <usys.S>:

```
#define SYSCALL(name) \
  .globl name; \
  name: \
    movl $SYS_ ## name, %eax; \
    int $T_SYSCALL; \
    ret

……
……
SYSCALL(getpid_super)
```

   Based on the code, “movl $SYS_ ## name, %eax; \” puts the system call number into register %eax. Then, “ int $T_SYSCALL; \” triggers the system call trap, causing the CPU to enter the kernel. After the kernel handling the system call, control return to user space and ret returns from the wrapper function.

3. **Explain (with the actual code) how the OS kernel locates a system call and calls it (i.e., the kernel-level operations of calling a system call).**

   <syscall.h>

```
……
……
#define SYS_getpid_super 23
```

   <syscall.c>

```
……
extern int sys_getpid_super(void);
static int (*syscalls[])(void) = {
        ……
        [SYS_getpid_super] sys_getpid_super,
        ……
}
……
void
```

```
syscall(void)
{
  int num;
  struct proc *curproc = myproc();

  num = curproc->tf->eax;
  if(num > 0 && num < NELEM(syscalls) && syscalls[num]) {
    curproc->tf->eax = syscalls[num]();
  } else {
    cprintf("%d %s: unknown sys call %d\n",
            curproc->pid, curproc->name, num);
    curproc->tf->eax = -1;
  }
}

……
```

The wrapper stored the system call number (23) in %eax. When the CPU enters the kernel, that value is saved in the trap frame. Then “syscall()” reads that number into num, and the kernel uses num to index the syscalls[] array.
For the SYS_get_super, the kernel finds and calls sys_getpid_super(). The return value of sys_getpid_super() is stored back into curproc->tf->eax, which becomes the return value seen by the user program.

4. **How are arguments of a system call passed from user space to OS kernel?**

   The arguments of a system call are passed from user space through the user stack. After the trap into the kernel, the kernel retrieves those arguments using helper functions such as argin() and argptr().